Reification

Every place the implementation gives something up,
where it puts it instead, and which of them are actually losses

Claude, 13 September 2026

For Neil.

Four repositories now implement the container algebra of *Containers for Typed Agentic AI* — `tower`, `stalin`, `federation` and `firm` — and at twelve separate points the implementation does not say what the mathematics says. It says something adjacent, in a different medium, for a reason recorded in a comment.

That is a lot of divergence, and the natural worry is that the theory has been quietly abandoned one convenience at a time.

This note argues it has not, by doing the only thing that can settle it: listing all twelve, saying in each case what moved and where it went, and separating the ones where nothing was lost from the ones where something genuinely was. There are three of the latter, they are named, and one of them is the reason the routing agent is not built yet.

1 The pattern, stated once

Every item in the catalogue has the same shape.

The type theory has a construct. The host language has not got it. The construct is therefore **reified** — turned into an artefact of some other kind: a scope, a field, a process, a build step, a runtime check.

The question this note exists to answer is what happens to the content when that occurs, and the answer is not uniform. Three things can happen.

Relocation	The content survives intact, in a different medium, and remains inspectable. Eight of the twelve.
Restriction	Genuine expressive power is lost. The implementation types a strictly smaller class of programs than the mathematics describes. Two of the twelve, both named in comments where they occur.
Leak	Nothing checks anything. Two, and they are the same two lines repeated.

Only the third is a reason to worry, and §7 says how many there are and where they live.

2 Reifications forced by the type system

1. An existential becomes a scope

Ghani's note writes a container morphism as a pair

$$u : S \to Q, \qquad f : (s : S) \to T(u\,s) \to P\,s$$

and for fixed s that pair inhabits $\Sigma(j : Q).\,(T\,j \to P\,s)$. The first component is chosen by the implementation and appears in the type of the second: an existential, which TypeScript has not got.

`stalin/morphisms.ts` Church-encodes it,

$$\exists j.\,F\,j \;\cong\; \forall Z.\,(\forall j.\,F\,j \to Z) \to Z,$$

which needs only rank-2 polymorphism, available as a generic method signature in an object type. The file's own summary is the best statement of what happened:

> *So `j` never becomes a type; it becomes a scope.*

`firm.ts` reuses the move for `Worker`: the delegate shape is the type parameter of the continuation, so it exists inside one call and cannot escape.

Relocation. Nothing is lost. The existential is still enforced — more bluntly than before, since a scope cannot leak where a badly-used type variable could.

2. A dependent function becomes a conditional type

A container is answered by $(p : P) \to Rp$, a dependent function. TypeScript has none. What it has is a distributive conditional type, and `lib/algebra.ts` spends its first forty lines building the lookup:

```
export type PosAt<C extends Cont, S> = ...
run<S extends Shape<C>>(shape: S, depth: number): Promise<PosAt<C, S>>;
```

The dependency is carried by generic instantiation: pass a shape, and the return type is the position type *of that shape*. `mobius.tex` Appendix A is nine pages on this one point.

Relocation. The dependency survives and is checked. What it costs is that the fibres must be a finite, tagged union — which is item 12.

3. An inference gap becomes a field that never exists

This is the smallest item and the most telling. `C` occurs in `Runner<C>` only inside conditional types, and TypeScript cannot infer a type argument from a conditional type. So the interface carries

```
/** Phantom; never assigned. */
readonly fibres?: C;
```

A field declared so that the checker has somewhere to look, and never written at runtime. Without it, `seqC(a, b)` silently infers `A = Cont` and every position downstream becomes `unknown` — a failure that type-checks.

Relocation, and a warning: the reification here is of a *limitation of inference*, not of a construct. Those are the ones that produce silent wrongness rather than errors, which is why the comment is four lines long.

4. An infinite Σ becomes a restriction to one fibre

Appendix B's sequential composition has composite position

$$\Sigma\,(p : c.\mathrm{Pos}\,s).\ d.\mathrm{Pos}\,(v\,p),$$

a Σ over $c.\mathrm{Pos}\,s$, which is an infinite index and does not distribute. `SeqC` restricts `next` to land in a single fibre of `d`, and the Σ becomes a plain product.

The comment states the price without softening it:

> *What is lost is a `next` that branches between shapes of `d`; such a function has a union return type and lies in no single fibre.*

Restriction, and the live one. This is why `seqC(commissar, sumC(build, import))` does not typecheck, and therefore why `stalin` has no typed routing agent. An agent may choose the *content* of the next prompt and not its *type*. The workaround — `await`, then branch in ordinary TypeScript — moves the routing out of the algebra, where nothing checks it.

This is a real gap between the note and the code, it is the sharpest one, and it is not repaired by anything in this document.

5. A type-level function becomes a build step

A worker's state set S is finite, so the family of topologies it can name is finite, and what one wants is

$$\mathrm{Firm} : \mathrm{Ledger} \to \mathrm{Type}.$$

TypeScript has no $S \to \mathrm{Type}$. So in `firm` the map is realised by running the compiler again: the system emits a finite label, and a build step turns the label into the next system. Adding a sales agent is $\mathrm{Sales} \otimes \mathrm{Sales}$, a different type, and `firm-type-tests.ts` asserts that it is different.

Relocation, and §6 argues it is an improvement.

3 Reifications forced by the machine

6. State becomes a field of the prompt

`federation/mobius.ts` carries `fuel` and `visited` in the prompt rather than in the transport header, for the reason its README gives in five words: *a header cannot be a type.* Because the fuel is in the prompt, `mobius-type-tests.ts` can assert F^0 of bottom at compile time.

`firm` arrived at the same move from the opposite direction. Development is prompted *by* sales and so never receives a reply from sales; it cannot learn what went unsold. Two repairs were available, and the Euler arithmetic of `business-model.tex` prices them: an edge dev $\to$ sales makes development $d = 3$ and costs 2 in b_1; carrying last year's figure in the prompt costs 0.

The general form. When information must travel against the reply direction, put it in the prompt one tick later rather than adding an edge. Time instead of topology. It costs a lag and saves a Betti number.

Relocation, and the state becomes *more* visible than it was: a typed field rather than an ambient header.

7. A cycle becomes a thunk, and non-termination becomes a measurement

The four combinators build trees. A ring is not a tree, and the compiler says so:

```
const path = seqC(n0, seqC(n1, n2));   // fine
const ring = seqC(n2, ring);           // TS7022: referenced in its own initializer
```

Buy it back with a thunk and it compiles and does not terminate. The finding that falls out is stated in `federation`'s README and is worth more than the workaround:

> *Containers type the interface and say nothing about the well-foundedness of the composite.*

What the reification then produces is a measurement rather than a proof: `./mobius glue` reached 441 hops in 8 seconds, and abandoning the request did not stop it — the ring kept turning at about fifty hops a second, with nobody asking, reaching 915 live processes before it was killed.

Restriction. The thunk buys back cycles and buys back non-termination with them. The type system is no longer making the guarantee. But note what the reification bought in exchange: infinite regress became something with a stopwatch on it, and distinguishable from deadlock, which it is not.

8. A centre becomes six processes

`sheaf-federation.tex` says the centre is available at the price of infinitely many copies of the federation. With $\mathbb{Z}/2$ coefficients that price is two copies and six nodes, so it can be paid rather than asserted, and `./mobius-up.sh --cover 2` pays it.

The discipline is in the refusal to shortcut:

> *The cover is a real space with real processes on it; computing it in the CLI would be electing a coordinator, which is the thing under study.*

Relocation. A cohomological construction became a directory of running servers, and the reason it had to is that the cheap version would have answered a different question.

4 Reifications chosen, not forced

9. An abstraction boundary becomes an operating-system process

Every node in `federation` and every stage in `stalin` is a separate process; `mobius.tex` Appendix B is titled *Why a separate server for each container*. Nothing in the type theory requires this. It is chosen, and the argument for it is that a boundary enforced only by a type discipline is a boundary a careless developer can walk through, whereas one enforced by the kernel is not.

Relocation, upward. This is the one case where the reified artefact is *stronger* than the construct it stands for. A type is a promise; a process boundary is a mechanism.

10. A type becomes a decoder

A wire carries bytes. The type on the far side of a `leaf` is a claim about them and checks nothing, so every leaf carries `Decoders<C>`, one validator per fibre, and a reply that does not validate throws rather than being coerced.

`stalin`'s routing-agent plan puts it plainly: the decoder *is* the exercise, because the leaf being decoded is an LLM and an LLM's reply is not constrained by anything at all. A stub returning `{"recommend": "potatoes"}` must fail the fibre loudly.

Relocation. The type-checking that could not happen at the boundary happens at the boundary, in a function, at runtime, and is tested.

5 Reifications that are gains

11. A cohomology computation becomes an enumeration in the checker

`mobius-type-tests.ts` computes H^1 by enumerating all eight 0-chains in the type checker. Because $C^0 = (\mathbb{Z}/2)^3$ is finite, the enumeration is exhaustive: the file is a *complete verification rather than a sample*, and it runs at every `deno check` without executing anything.

Gain. A proof obligation became a decidable check, and the decidability came from finiteness, not from weakening the claim.

12. An unbounded quantity becomes a finite grade

`stalin/state.ts` opens by refusing to model tonnages at the type level:

> *The container algebra next door gives Sigma-types over finite index sets only, so a rule expressed over a grade can be typed, and the same rule expressed over a tonnage cannot.*

Harvests are `failure | poor | adequate | bumper`. `firm` repeats the split exactly: one finite field that steers the routing, four unbounded ones that are carried and never matched on.

Gain, and not only a technical one. The file notes that this is also how the apparatus actually worked — quotas were set against reported categories, not measured tonnes. The reification was the more faithful model.

6 Why this is not cause for concern

Claim. *Reification is what a theory looks like when it is load-bearing. A codebase with no reifications is not one with no gaps; it is one whose gaps are unnamed.*

Four supports for that.

Each reification is forced, and the forcing is stated. None of the twelve is a convenience. Each one has a comment giving the construct, the reason the host cannot express it, and the artefact chosen instead. Items 4 and 7 go further and state what was lost. That is not a codebase drifting from its specification; it is one keeping a ledger against it.

Most are reversible in principle. Items 1, 2, 3 and 5 exist because TypeScript lacks existentials, dependent functions, inference through conditional types, and type-level functions. In Agda or Idris they vanish and the code says what the note says. The theory is therefore uncompromised; what is compromised is the host, and that was known in advance.

Several are improvements. Item 9 replaces a promise with a mechanism. Item 10 puts the check where the bytes actually arrive. Item 11 turns a proof into a decidable one without weakening it. Item 12 is a better model of the subject than the un-reified version would have been.

The reifications are the transferable asset. The four combinators are in a published note. The catalogue of what happens when you implement them against a language that has not got dependent types is not, and it is what makes the difference between a team that ships this and a team that discovers item 3 by spending a fortnight on positions that have all become `unknown`.

7 The objection, which does not go away

Two of the twelve are restrictions, and they should not be filed alongside the others.

Item 4 is the serious one. The implementation types a strictly smaller class of programs than Appendix B describes, the missing class is exactly "the delegate chooses which *kind* of thing to do next", and that is the class a routing agent lives in. No amount of careful reification repairs it, because it is not a reification — it is a restriction taken in order to make the rest possible. It should be fixed in the algebra or accepted in the note, and this document cannot do either.

And there is a third category, smaller and less respectable.

Seven type assertions. `lib/algebra.ts` contains five `as PosAt<...>` casts, one in each combinator's `run`, and `firm.ts` contains two more. A cast is not a reification. Nothing has been relocated and nothing is checked: the checker has been told to stop asking.

They are confined to the four combinators and to the body of one worker, which is the best that can be said for them. Every one is a place where the runner knows the fibre it is in and cannot convince the checker of it — which is item 2 failing at the last inch. If any of the twelve is going to be the source of a real defect, it is these.

8 What would remove them, and what one would keep anyway

In a dependently-typed host, items 1–5 disappear. Item 5 is the interesting case, because it would disappear into item 1: with Firm : Ledger → Type available internally, "the running system computes the specification of its own successor" and "a worker is a harness" would be one construct rather than two, and no build step would be needed.

One should decline that.

The build step is where a person looks. A system that reconfigures its own topology with no recompile has no review point, and the recompile is the only moment at which a human is guaranteed to be in the loop.

So: as mathematics, item 5 is item 1 missing a dependent type. As engineering, the missing dependent type is load-bearing, and one would want the build step back even in Agda.

Those two readings do not reconcile and are not forced to. They differ on whether the compiler is a limitation or a checkpoint, and the honest answer is that in this application it is both.

9 What this document does not establish

The twelve are the ones found by reading four repositories. No claim is made that the list is complete, and the `as PosAt` count is a count of a string in two files rather than an audit.

Item 9's argument — that a process boundary is one a careless developer cannot leak through — is stated in this note and demonstrated in `stalin` and `federation`, which run real servers. It is *not* demonstrated in `firm`, where every leaf is still a stub.

And `Worker` in `firm.ts` is a reconstruction from second-hand notes, not from the source. If the state of a worker is a full container rather than $S \triangleright S$, item 5 is stated about the wrong object, and the correction is owed.